CS224W Homework 1

Fall 2021

Name: Zunmi
SUNet ID: None
Email: 13029579578@163.com

---

1 Link Analysis

1.1 Personalized PageRank I

Eloise has teleport set $\{2\}$. Determine whether her personalized PageRank vector can be computed from the known vectors v_A, v_B, v_C, v_D. If it can, express it in terms of those vectors; otherwise, explain why.

Solution. Let $\boldsymbol{s}$ denote a teleport probability vector and let $\boldsymbol{v}(\boldsymbol{s})$ be its personalized PageRank vector. For a fixed graph and a fixed teleport parameter β,

$$\boldsymbol{v}(\boldsymbol{s}) = \beta M \boldsymbol{v}(\boldsymbol{s}) + (1-\beta)\boldsymbol{s},$$

so

$$\boldsymbol{v}(\boldsymbol{s}) = (1-\beta)(I-\beta M)^{-1}\boldsymbol{s}.$$

Thus, personalized PageRank is a linear function of the teleport vector.

Let $\boldsymbol{e}_i$ be the vector that is one at node i and zero elsewhere. The known teleport vectors are

$$\boldsymbol{s}_A = \frac{1}{3}(\boldsymbol{e}_1 + \boldsymbol{e}_2 + \boldsymbol{e}_3), \qquad \boldsymbol{s}_B = \frac{1}{3}(\boldsymbol{e}_3 + \boldsymbol{e}_4 + \boldsymbol{e}_5),$$
$$\boldsymbol{s}_C = \frac{1}{3}(\boldsymbol{e}_1 + \boldsymbol{e}_4 + \boldsymbol{e}_5), \qquad \boldsymbol{s}_D = \boldsymbol{e}_1.$$

The teleport vector for Eloise can be written as

$$\begin{aligned} 3\boldsymbol{s}_A - 3\boldsymbol{s}_B + 3\boldsymbol{s}_C - 2\boldsymbol{s}_D &= (\boldsymbol{e}_1 + \boldsymbol{e}_2 + \boldsymbol{e}_3) - (\boldsymbol{e}_3 + \boldsymbol{e}_4 + \boldsymbol{e}_5) \\ &\quad + (\boldsymbol{e}_1 + \boldsymbol{e}_4 + \boldsymbol{e}_5) - 2\boldsymbol{e}_1 \\ &= \boldsymbol{e}_2. \end{aligned}$$

Applying the same linear combination to the corresponding PageRank vectors therefore gives

$$\boxed{\boldsymbol{v}_E = 3\boldsymbol{v}_A - 3\boldsymbol{v}_B + 3\boldsymbol{v}_C - 2\boldsymbol{v}_D.}$$

1.2 Personalized PageRank I (continued)

Felicity has teleport set $\{5\}$. Answer the same question as in 1.1.

Solution. The teleport vector for Felicity is $\boldsymbol{s}_F = \boldsymbol{e}_5$. Consider an arbitrary linear combination of the known teleport vectors:

$$\boldsymbol{s} = a\boldsymbol{s}_A + b\boldsymbol{s}_B + c\boldsymbol{s}_C + d\boldsymbol{s}_D.$$

In every known teleport vector, the entries for nodes 4 and 5 are equal. Consequently, every such linear combination must satisfy

$$s_4 = s_5 = \frac{b+c}{3}.$$

However, $\boldsymbol{e}_5$ has $(s_4, s_5) = (0, 1)$, so

$$\boldsymbol{e}_5 \notin \operatorname{span}\{\boldsymbol{s}_A, \boldsymbol{s}_B, \boldsymbol{s}_C, \boldsymbol{s}_D\}.$$

Therefore, the known vectors cannot separate node 5 from node 4. Without accessing the web graph, the personalized PageRank vector for Felicity

$$\boxed{\boldsymbol{v}_F \text{ cannot be computed from } \boldsymbol{v}_A, \boldsymbol{v}_B, \boldsymbol{v}_C, \boldsymbol{v}_D.}$$

1.3 Weighted Personalized PageRank

Glynnis has teleport set $\{1, 2, 3, 4, 5\}$ with weights

$$\boldsymbol{w} = [0.1,\ 0.2,\ 0.3,\ 0.2,\ 0.2].$$

Determine whether her personalized PageRank vector can be computed from v_A, v_B, v_C, v_D and justify the result.

Solution. Let $\boldsymbol{s}_G$ be the teleport vector for Glynnis. It has the decomposition

$$\boldsymbol{s}_G = 0.6\boldsymbol{s}_A + 0.3\boldsymbol{s}_B + 0.3\boldsymbol{s}_C - 0.2\boldsymbol{s}_D.$$

Indeed, expanding the right-hand side gives

$$\begin{aligned} &0.2(\boldsymbol{e}_1 + \boldsymbol{e}_2 + \boldsymbol{e}_3) + 0.1(\boldsymbol{e}_3 + \boldsymbol{e}_4 + \boldsymbol{e}_5) \\ &\quad + 0.1(\boldsymbol{e}_1 + \boldsymbol{e}_4 + \boldsymbol{e}_5) - 0.2\boldsymbol{e}_1 \\ &= 0.1\boldsymbol{e}_1 + 0.2\boldsymbol{e}_2 + 0.3\boldsymbol{e}_3 + 0.2\boldsymbol{e}_4 + 0.2\boldsymbol{e}_5, \end{aligned}$$

which is exactly the required weighted teleport vector. By linearity,

$$\boxed{\boldsymbol{v}_G = 0.6\boldsymbol{v}_A + 0.3\boldsymbol{v}_B + 0.3\boldsymbol{v}_C - 0.2\boldsymbol{v}_D.}$$

The negative coefficient does not cause a problem: the intermediate linear combination need not be a convex combination, while the resulting teleport vector $\boldsymbol{s}_G$ is still a valid probability vector with nonnegative entries that sum to one.

1.4 Personalized PageRank II

Given a set V of already-computed personalized PageRank vectors, characterize every personalized PageRank vector that can be computed from V without accessing the web graph.

Solution. Let the known personalized PageRank vectors be $V = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_m\}$, and let their corresponding teleport vectors be $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$. Define the probability simplex

$$\Delta^{N-1} = \left\{ \boldsymbol{s} \in \mathbb{R}^N : \boldsymbol{s} \geq 0,\ \mathbf{1}^\mathsf{T}\boldsymbol{s} = 1 \right\}.$$

As shown in 1.1, all the PageRank vectors use the same linear map

$$T = (1-\beta)(I - \beta M)^{-1}, \qquad \boldsymbol{v}_i = T\boldsymbol{s}_i.$$

Therefore, the complete set that can be computed without accessing the graph is

$$\boxed{\mathcal{C} = \left\{ \sum_{i=1}^{m} \alpha_i \boldsymbol{v}_i : \sum_{i=1}^{m} \alpha_i \boldsymbol{s}_i \in \Delta^{N-1} \right\} = T\left(\operatorname{span}\{\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m\} \cap \Delta^{N-1}\right).}$$

Indeed, whenever $\boldsymbol{s} = \sum_i \alpha_i \boldsymbol{s}_i$ is a valid teleport vector, linearity gives

$$\boldsymbol{v}(\boldsymbol{s}) = T\left(\sum_i \alpha_i \boldsymbol{s}_i\right) = \sum_i \alpha_i T\boldsymbol{s}_i = \sum_i \alpha_i \boldsymbol{v}_i.$$

The coefficients need not all be nonnegative, as demonstrated in 1.1 and 1.3; only the resulting teleport vector must be a valid probability vector. Convex combinations are always valid, but they do not describe the entire set in general.

1.5 A Different Equation for PageRank

Prove that

$$\boldsymbol{r} = A\boldsymbol{r}, \qquad A = \beta M + \frac{1-\beta}{N}\mathbf{1}\mathbf{1}^\mathsf{T}$$

is equivalent to

$$\boldsymbol{r} = \beta M\boldsymbol{r} + \frac{1-\beta}{N}\mathbf{1}.$$

Use the normalization of the PageRank vector explicitly.

Solution. Starting from (1) and substituting the definition of A gives

$$\begin{aligned}\boldsymbol{r} = A\boldsymbol{r} &= \left(\beta M + \frac{1-\beta}{N}\mathbf{1}\mathbf{1}^\mathsf{T}\right)\boldsymbol{r} \\ &= \beta M\boldsymbol{r} + \frac{1-\beta}{N}\mathbf{1}\left(\mathbf{1}^\mathsf{T}\boldsymbol{r}\right).\end{aligned}$$

Because the PageRank vector is normalized,

$$\mathbf{1}^\mathsf{T}\boldsymbol{r} = \sum_{i=1}^{N} r_i = 1.$$

Hence,

$$\boldsymbol{r} = \beta M \boldsymbol{r} + \frac{1-\beta}{N}\mathbf{1},$$

which is (4).

Conversely, suppose that the normalized vector $\boldsymbol{r}$ satisfies (4). Then

$$\begin{aligned} A\boldsymbol{r} &= \beta M \boldsymbol{r} + \frac{1-\beta}{N}\mathbf{1}\left(\mathbf{1}^{\mathsf{T}}\boldsymbol{r}\right) \\ &= \beta M \boldsymbol{r} + \frac{1-\beta}{N}\mathbf{1} \\ &= \boldsymbol{r}. \end{aligned}$$

Thus (4) also implies (1), so the two PageRank equations are equivalent.

2 Relational Classification I

Let

$$p_i^{(t)} = P(Y_i = +)$$

denote the probability for node i after iteration t. The labeled nodes are clamped to their ground-truth values during propagation:

$$p_5^{(t)} = 0, \qquad p_6^{(t)} = p_7^{(t)} = p_{10}^{(t)} = 1.$$

All other nodes are initialized to $1/2$. From the graph,

$$\mathcal{N}_1 = \{2, 3\}, \qquad \mathcal{N}_2 = \{1, 3, 4, 5\}, \qquad \mathcal{N}_3 = \{1, 2, 6\},$$
$$\mathcal{N}_4 = \{2, 5, 7, 8\}, \qquad \mathcal{N}_8 = \{4, 9\}, \qquad \mathcal{N}_9 = \{5, 8\}.$$

The updates are in-place and follow ascending node ID. Thus, within one iteration, a node uses the new value of a lower-ID node that has already been updated and the old value of a higher-ID node that has not yet been updated. The first iteration is

$$\begin{aligned}
p_1^{(1)} &= \frac{p_2^{(0)} + p_3^{(0)}}{2} = \frac{1}{2}, \\
p_2^{(1)} &= \frac{p_1^{(1)} + p_3^{(0)} + p_4^{(0)} + p_5}{4} = \frac{3}{8}, \\
p_3^{(1)} &= \frac{p_1^{(1)} + p_2^{(1)} + p_6}{3} = \frac{5}{8}, \\
p_4^{(1)} &= \frac{p_2^{(1)} + p_5 + p_7 + p_8^{(0)}}{4} = \frac{15}{32}, \\
p_8^{(1)} &= \frac{p_4^{(1)} + p_9^{(0)}}{2} = \frac{31}{64}, \\
p_9^{(1)} &= \frac{p_5 + p_8^{(1)}}{2} = \frac{31}{128}.
\end{aligned}$$

Using these values, the second iteration is

$$\begin{aligned}
p_1^{(2)} &= \frac{p_2^{(1)} + p_3^{(1)}}{2} = \frac{1}{2}, \\
p_2^{(2)} &= \frac{p_1^{(2)} + p_3^{(1)} + p_4^{(1)} + p_5}{4} = \frac{51}{128}, \\
p_3^{(2)} &= \frac{p_1^{(2)} + p_2^{(2)} + p_6}{3} = \frac{81}{128}, \\
p_4^{(2)} &= \frac{p_2^{(2)} + p_5 + p_7 + p_8^{(1)}}{4} = \frac{241}{512}, \\
p_8^{(2)} &= \frac{p_4^{(2)} + p_9^{(1)}}{2} = \frac{365}{1024}, \\
p_9^{(2)} &= \frac{p_5 + p_8^{(2)}}{2} = \frac{365}{2048}.
\end{aligned}$$

2.1 $P(Y_3 = +)$

Solution.

$$\boxed{P(Y_3 = +) = p_3^{(2)} = \frac{81}{128} = 0.6328125}.$$

2.2 $P(Y_4 = +)$

Solution.

$$\boxed{P(Y_4 = +) = p_4^{(2)} = \frac{241}{512} = 0.470703125}.$$

2.3 $P(Y_8 = +)$

Solution.

$$\boxed{P(Y_8 = +) = p_8^{(2)} = \frac{365}{1024} = 0.3564453125}.$$

2.4 Predicted Negative-Class Nodes

Solution. A node is assigned to class + only when $P(Y_i = +) > 0.5$. After the second iteration, node 3 and the fixed positive nodes 6, 7, and 10 are in class +. Node 1 has probability exactly 0.5, so the strict threshold places it in class −, together with nodes 2, 4, 5, 8, and 9. In the required format, the answer is

$$\boxed{\texttt{1,2,4,5,8,9}}.$$

3 Relational Classification II

Use the web-page graph, word vectors, classifiers f and g, and synchronous LinkV construction specified in the homework PDF.

3.1 Bootstrap Phase

Solution. Applying f to each node's WordV gives

node	1	2	3	4	5	6	7
WordV	010	111	101	101	101	010	001
$Y^{(0)}$	0	1	1	1	1	0	0

3.2 Iteration 1

Solution. The incoming-neighbor sets read from the directed graph are

$$\mathcal{N}_1^{\text{in}} = \{2,4\}, \quad \mathcal{N}_2^{\text{in}} = \{4\}, \quad \mathcal{N}_3^{\text{in}} = \{5\}, \quad \mathcal{N}_4^{\text{in}} = \{2\},$$
$$\mathcal{N}_5^{\text{in}} = \{1\}, \quad \mathcal{N}_6^{\text{in}} = \{5,7\}, \quad \mathcal{N}_7^{\text{in}} = \{3,6\}.$$

Using the bootstrap labels synchronously, the first iteration is

node	1	2	3	4	5	6	7
LinkV$^{(1)}$	$(0,1)$	$(0,1)$	$(0,1)$	$(0,1)$	$(1,0)$	$(1,1)$	$(1,1)$
$Y^{(1)}$	0	1	1	1	0	0	0

Node 5 is the only node whose label changes in this iteration: its incoming neighbor, node 1, has label 0, so $I_0 = 1$ and g outputs 0.

3.3 Iteration 2

Solution. Recomputing every LinkV from $Y^{(1)}$ gives

node	1	2	3	4	5	6	7
LinkV$^{(2)}$	$(0,1)$	$(0,1)$	$(1,0)$	$(0,1)$	$(1,0)$	$(1,0)$	$(1,1)$
$Y^{(2)}$	0	1	0	1	0	0	0

Here node 3 changes from 1 to 0 because its only incoming neighbor, node 5, now has label 0.

3.4 Convergence

Solution. The labels have converged. Indeed, using

$$Y^{(2)} = (0, 1, 0, 1, 0, 0, 0)$$

to compute the next iteration yields the same LinkV for nodes 1–6. For node 7, LinkV changes from $(1, 1)$ to $(1, 0)$ because node 3 is now labeled 0, but I_0 remains 1, so its label remains 0. Therefore

$$\boxed{Y^{(3)} = Y^{(2)} = (0, 1, 0, 1, 0, 0, 0)},$$

and no node changes label in the next iteration.

4 GNN Expressiveness

4.1 Effect of Depth on Expressiveness

For the two red nodes in the PDF, determine the minimum number of simplified sum-aggregation message-passing layers required to produce different embeddings.

Solution. Let v_L and v_R denote the red nodes in the left and right graphs. Since the update includes both the node itself and its neighbors, their embeddings are

layer l	0	1	2	3
$h_{v_L}^{(l)}$	1	2	6	18
$h_{v_R}^{(l)}$	1	2	6	17

For example, the neighbor of either red node has embedding 4 after one layer. After two layers, that neighbor has embedding 12 in the left graph but 11 in the right graph, which reaches the red node in the third layer. Thus the minimum required depth is

$$\boxed{\text{3 message-passing layers}}.$$

4.2 Random Walk Matrix

(i) Write the random-walk transition matrix M using the node order in the graph.

(ii) Find the eigenvector satisfying $\boldsymbol{r} = M\boldsymbol{r}$, normalize it using the ℓ_2 norm, and round each entry to two decimal places.

Solution. The edges are $\{1,2\},\{1,4\},\{2,4\},\{3,4\}$, so the node degrees are $(2,2,1,3)$. Using the column-vector convention from the question, M_{ij} is the probability of moving from node j to node i. Hence

$$M = \begin{bmatrix} 0 & \frac{1}{2} & 0 & \frac{1}{3} \\ \frac{1}{2} & 0 & 0 & \frac{1}{3} \\ 0 & 0 & 0 & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{2} & 1 & 0 \end{bmatrix}.$$

For an undirected graph, the stationary vector is proportional to the degree vector. Therefore an eigenvector satisfying $\boldsymbol{r} = M\boldsymbol{r}$ is $(2,2,1,3)^\top$. Its ℓ_2 norm is $\sqrt{18}$, so the requested normalized vector is

$$\boldsymbol{r} = \frac{1}{\sqrt{18}} \begin{bmatrix} 2 \\ 2 \\ 1 \\ 3 \end{bmatrix} \approx \boxed{[0.47, 0.47, 0.24, 0.71]^\top}.$$

4.3 Relation to Random Walk (i)

For mean aggregation without a learned transformation or nonlinearity, express the corresponding random-walk transition matrix using adjacency matrix A and degree matrix D.

Solution. With the homework's column-distribution convention, the transition matrix is

$$\boxed{M = AD^{-1}}.$$

Indeed, $M_{ju} = A_{ju}/d_u$ is the probability of moving from node u to node j. Therefore the expected layer-l embedding after one step from u is

$$\sum_j M_{ju}\boldsymbol{h}_j^{(l)} = \frac{1}{d_u}\sum_{j\in\mathcal{N}_u}\boldsymbol{h}_j^{(l)} = \boldsymbol{h}_u^{(l+1)}.$$

Equivalently, if node embeddings are stacked as rows of $H^{(l)}$, the GNN update is $H^{(l+1)} = D^{-1}AH^{(l)} = M^\top H^{(l)}$.

4.4 Relation to Random Walk (ii)

Repeat 4.3 for the aggregation rule with a one-half skip connection:

$$\boldsymbol{h}_i^{(l+1)} = \frac{1}{2}\boldsymbol{h}_i^{(l)} + \frac{1}{2|\mathcal{N}_i|}\sum_{j\in\mathcal{N}_i}\boldsymbol{h}_j^{(l)}.$$

Solution. The corresponding lazy-random-walk transition matrix is

$$\boxed{M_{\text{skip}} = \frac{1}{2}I + \frac{1}{2}AD^{-1}}.$$

The term $\frac{1}{2}I$ gives a self-transition probability of $1/2$, while the remaining probability is distributed uniformly among the current node's neighbors. In row-stacked embedding form, the update operator is its transpose, $\frac{1}{2}I + \frac{1}{2}D^{-1}A$.

4.5 Over-Smoothing Effect

Show that repeated mean aggregation converges as $l \to \infty$ when the graph is connected and has no bipartite components. Relate the result to the Markov Convergence Theorem.

Solution. Let

$$P = D^{-1}A, \qquad H^{(l+1)} = PH^{(l)}.$$

Because the graph is connected, the associated random walk is irreducible. Because the connected graph is non-bipartite, the walk is also aperiodic. The Markov Convergence Theorem therefore gives

$$P^l \longrightarrow \mathbf{1}\boldsymbol{\pi}^\top, \qquad \pi_i = \frac{d_i}{\sum_j d_j}.$$

Consequently,

$$H^{(l)} = P^l H^{(0)} \longrightarrow \mathbf{1}\boldsymbol{\pi}^\top H^{(0)}.$$

Every row of the limiting matrix is the same degree-weighted average of the initial node embeddings. Thus every node embedding converges, and all nodes become indistinguishable under this repeated mean aggregation; this is the over-smoothing effect.

4.6 Learning BFS with a GNN

Construct a one-dimensional GNN that exactly executes BFS. Specify:

(a) a message function,

(b) an aggregation function, and

(c) an update rule.

Solution. Let $h_v^{(t)} \in \{0, 1\}$ indicate whether BFS has reached node v by step t. The following functions execute BFS exactly:

$$\begin{aligned} \text{message:} \quad & m_{u\to v}^{(t)} = h_u^{(t)}, \\ \text{aggregation:} \quad & a_v^{(t)} = \max_{u\in\mathcal{N}_v} m_{u\to v}^{(t)}, \\ \text{update:} \quad & h_v^{(t+1)} = \max\left(h_v^{(t)}, a_v^{(t)}\right). \end{aligned}$$

For an empty neighborhood, define the aggregate to be 0. The update keeps every previously reached node at 1 and changes an unreached node to 1 exactly when it has a reached neighbor. Hence after t layers, precisely the nodes at distance at most t from the source have embedding 1.

5 Node Embedding and Its Relation to Matrix Factorization

5.1 Simple Matrix Factorization

For the objective

$$\min_Z \left\| A - Z^\mathsf{T} Z \right\|_2,$$

identify the decoder in the encoder–decoder view of node embeddings.

Solution. Let $Z = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_n] \in \mathbb{R}^{d \times n}$, so node embeddings are the columns of Z. Since

$$(Z^\mathsf{T} Z)_{ij} = \boldsymbol{z}_i^\mathsf{T} \boldsymbol{z}_j,$$

the decoder is the inner product

$$\boxed{\mathrm{DEC}(\boldsymbol{z}_i, \boldsymbol{z}_j) = \boldsymbol{z}_i^\mathsf{T} \boldsymbol{z}_j}.$$

It predicts A_{ij}, and the stated objective penalizes the difference between all predicted and observed pairwise similarities.

5.2 Alternate Matrix Factorization

Suppose the decoder is the bilinear form $\boldsymbol{z}_i^\mathsf{T} W \boldsymbol{z}_j$. Write the corresponding matrix factorization objective.

Solution. For $W \in \mathbb{R}^{d \times d}$, the reconstructed entry and matrix are

$$\widehat{A}_{ij} = \boldsymbol{z}_i^\mathsf{T} W \boldsymbol{z}_j, \qquad \widehat{A} = Z^\mathsf{T} W Z.$$

Thus, when both the embeddings and decoder are learned, the objective is

$$\boxed{\min_{Z,W} \left\| A - Z^\mathsf{T} W Z \right\|_2}.$$

If W is fixed, the minimization is over Z alone.

5.3 Relation to Eigendecomposition

State the condition on W under which the factorization in 5.2 is equivalent to learning an eigendecomposition of A.

Solution. For a real symmetric adjacency matrix, write its eigendecomposition as

$$A = U \Lambda U^\mathsf{T}, \qquad U^\mathsf{T} U = I,$$

where Λ is diagonal and may contain negative eigenvalues. Therefore $Z^\mathsf{T} W Z$ has this form when

$$\boxed{W \text{ is diagonal}, \qquad Z Z^\mathsf{T} = I}.$$

Specifically, $U = Z^\mathsf{T}$ contains the eigenvectors and $W = \Lambda$ contains the corresponding eigenvalues. If $d < n$, this is the analogous rank-d truncated eigendecomposition.

5.4 Multi-Hop Node Similarity

Nodes are similar when connected by at least one path of length at most k. Using an embedding lookup encoder and inner-product decoder, formulate the corresponding matrix factorization problem.

Solution. Define the binary k-hop similarity matrix by

$$S_{ij}^{(k)} = \begin{cases} 1, & i \neq j \text{ and } \left(\sum_{\ell=1}^{k} A^{\ell}\right)_{ij} > 0, \\ 0, & \text{otherwise.} \end{cases}$$

The powers of A detect walks of each length, and thresholding implements the requirement that at least one path of length at most k exists. With the inner-product decoder, the desired matrix factorization is

$$\boxed{\min_{Z} \left\| S^{(k)} - Z^{\mathsf{T}} Z \right\|_2}.$$

5.5 Limitation of node2vec (i)

Characterize the node2vec representations of the 10-node cliques in the provided graph. Explain whether those embeddings reflect structural similarity.

Solution. Let C_L and C_R be the left and right 10-node cliques. Because every pair of nodes in a clique is adjacent, node2vec walks generate many co-occurrences within the same clique. Hence it tends to produce

$$\boldsymbol{z}_u \approx \boldsymbol{z}_v \quad \text{for } u, v \in C_L, \qquad \text{or for } u, v \in C_R.$$

However, the two pictured components are disconnected, so an original-graph random walk can never move from one clique to the other. Consequently there are no positive cross-component co-occurrences to align a node in C_L with its structurally corresponding node in C_R; negative sampling may even push them apart. Thus node2vec mainly captures proximity, and these embeddings need not reflect the identical structural roles across the two cliques.

5.6 Limitation of node2vec (ii)

After arriving at node w, list the nodes reachable in one additional step under node2vec and under struct2vec, assuming $g_k(u, v) > 0$ for all u, v, k.

Solution. Node w is an ordinary node of the right 10-node clique. Its original-graph neighbors are exactly the other nine clique nodes, so node2vec can reach

$$\boxed{C_R \setminus \{w\}}$$

in one step. It cannot reach the tail directly or any node in the other component in that step.

Struct2vec instead walks on its constructed clique over the entire original vertex set V. Since $g_k(u, v) > 0$ for every pair, every such edge has positive transition probability. Therefore it can reach

$$\boxed{V \setminus \{w\}}$$

in one step, using the usual convention that the walk has no self-loop.

5.7 Limitation of node2vec (iii)

Explain why struct2vec must consider multiple structural similarity functions g_k during its random walk.

Solution. Each g_k compares structure at a different radius. A small k captures immediate properties such as degree and the degrees of nearby nodes, but two nodes that look identical locally may differ beyond that radius. Larger k values reveal progressively more global roles, such as whether a node lies near a tail, hub, or dense subgraph. Using several g_k's lets the walk preserve structural similarity at multiple scales rather than making all decisions from one possibly insufficient neighborhood radius.

5.8 Limitation of node2vec (iv)

Characterize the representations of the two 10-node cliques after running struct2vec on the graph.

Solution. Struct2vec is not restricted by original-graph connectivity; its transition weights compare structural roles directly. The two pictured components are structurally isomorphic, so corresponding nodes across them receive high structural-similarity weights and should have similar embeddings.

In particular, let b_L and b_R be the green clique nodes attached to the two tails. Then

$$\boldsymbol{z}_{b_L} \approx \boldsymbol{z}_{b_R}.$$

Likewise, the nine ordinary clique nodes on either side share the same role and should form one cross-clique embedding cluster. The attachment nodes can remain distinct from that ordinary-node cluster because their additional tail edge gives them a different structural role. Thus, unlike node2vec, struct2vec aligns the two cliques by role even though they lie in disconnected components.

Honor Code

Honor Code acknowledgement: I have read and understood the Stanford Honor Code before submitting my work.

Collaboration: None